

Adivina Quién

Documentación técnica · Trabajo práctico obligatorio 1

Dato	Detalle
Asignatura	Programación III — Diseño y Análisis de Algoritmos
Docente	López, Juan Ignacio
Comisión	Miércoles, turno noche [COMPLETAR: número de comisión]
Integrantes	[COMPLETAR: apellido, nombre y legajo de cada integrante]
Fecha de entrega	[COMPLETAR]
Código fuente	Carpeta ZIP adjunta y repositorio github.com/nacho2112-nd/progra-3-miercoles-noche-(carpeta tpo-1-advina-quien)

Los campos marcados como **[COMPLETAR]** los completa el equipo antes de imprimir.

1. Introducción.....	3
2. Diagrama de clases (UML)	3
2.1 Estructura general del proyecto	3
2.2 Relaciones entre clases.....	4
2.3 Justificación del modelo de datos	7
3. Bitácora de desarrollo.....	8
3.1 Etapas	8
3.2 Herramientas	8
3.3 Uso de inteligencia artificial.....	8
3.4 División del trabajo	9
3.5 Problemas encontrados y cómo se resolvieron.....	9
3.6 Fragmentos de código y su complejidad	9
4. Justificación algorítmica de Divide y Conquista.....	16
4.1 Algoritmo de ordenamiento inicial.....	16
4.2 Elección entre MergeSort y QuickSort.....	17
4.3 Criterio de ordenamiento	17
4.4 Estrategia de selección del filtro.....	17
4.5 Función de evaluación del filtro	17
4.6 Relación con el árbol de decisión del juego.....	18
4.7 Cómo se descartan los candidatos en cada turno	19
4.8 Criterio de parada y suposición final	19
4.9 Manejo de la dependencia lógica	19
4.10 Precondiciones y validaciones implementadas	20
4.11 Tabla comparativa de tiempos	20
5. Justificación de la estrategia Greedy utilizada.....	21
5.1 Por qué elegimos esta variante	21
5.2 Cómo se calcula el descarte.....	21
5.3 Por qué es eficiente	21
5.4 ¿La estrategia es óptima?	22
6. Algoritmos no utilizados	22
7. Notación Big O para nuestra app.....	23
8. Reflexión sobre el TP	23
Logros	23
Dificultades	24
Propuestas de mejora.....	24
9. Bibliografía, fuentes y ayudas.....	24
Anexo · Capturas de la aplicación	25

1. Introducción

Adivina Quién es un juego de dos jugadores. Cada uno elige en secreto un personaje de un tablero común y, turno a turno, trata de descubrir el del rival: hace preguntas de sí o no, o arriesga un nombre. Nuestra implementación en Java tiene dos modos, **Jugador vs Máquina** y **Máquina vs Máquina**. En el segundo, la pantalla muestra todo el razonamiento de cada máquina.

Dividimos el problema en cuatro subsistemas y resolvimos cada uno con una estrategia distinta:

- **Armado del tablero (Divide y Conquista).** El catálogo llega agrupado sólo por género. La máquina valida las precondiciones, ordena los personajes por nombre con MergeSort y los agrega a una lista que les asigna un ID autoincremental. Como el tablero queda ordenado por nombre, cada arriesgue se resuelve con búsqueda binaria.
- **Motor de turnos.** La clase `Partida` alterna los turnos. A cada jugador le entrega a su rival sólo como un `Oraculo`: una interfaz que responde SÍ o NO y nunca devuelve el personaje secreto.
- **Estrategia de la máquina (Greedy).** En cada turno, la máquina evalúa los filtros sin resolver y elige el de mayor descarte garantizado, $\min(\text{SÍ}, \text{NO})$. Una base de conocimiento aplica la dependencia lógica entre filtros (por ejemplo, un pelado no tiene color de pelo). Un criterio de parada decide cuándo conviene arriesgar.
- **Visualización y verificación.** La interfaz Swing muestra los tableros, la evaluación de cada turno, la traza de MergeSort, el árbol de decisión completo y la comparación de tiempos. Hay además 54 pruebas automáticas y una herramienta que compara la estrategia con el óptimo exacto.

Las reglas, tal como las implementamos:

Regla	Decisión
Personajes	23, los mismos para los dos jugadores.
Filtros	Los de la consigna: género, calvicie (en el juego, "¿Es pelado?"), lentes y pelo colorado, negro o amarillo. Agregamos barba y sombrero (ver abajo).
Turno	Una sola acción: preguntar un filtro o arriesgar un personaje.
Arriesgue	Si acierta, gana. Si falla, pierde el turno y ese personaje queda descartado.
Respuestas	Automáticas: salen del secreto de cada jugador, a través del Oraculo. El humano no contesta a mano y no puede mentir.
Secretos	El humano elige el suyo haciendo clic en una carta. La máquina elige el suyo al azar.
Quién empieza	El humano en Jugador vs Máquina; la Máquina A en Máquina vs Máquina.

Por qué barba y sombrero. Con los filtros de la consigna hay $2 \times 4 \times 2 = 16$ combinaciones posibles: género, una de cuatro opciones de pelo (pelado, colorado, negro o amarillo) y lentes. Con 23 personajes, al menos 7 quedarían idénticos a otro y ninguna pregunta podría separarlos. Con barba (sólo en hombres) y sombrero hay 48 perfiles posibles, y los 23 personajes son distinguibles, como pide la consigna.

2. Diagrama de clases (UML)

2.1 Estructura general del proyecto

El código está en cinco paquetes, más una carpeta de pruebas. Las dependencias van en un solo sentido y no hay ciclos. La interfaz depende del motor; el motor, de los algoritmos y los datos; y todos, del modelo. Así se puede probar el motor sin abrir ninguna ventana, y cambiar la interfaz sin tocar la lógica.

Estructura del proyecto · dependencias entre paquetes (sin ciclos)

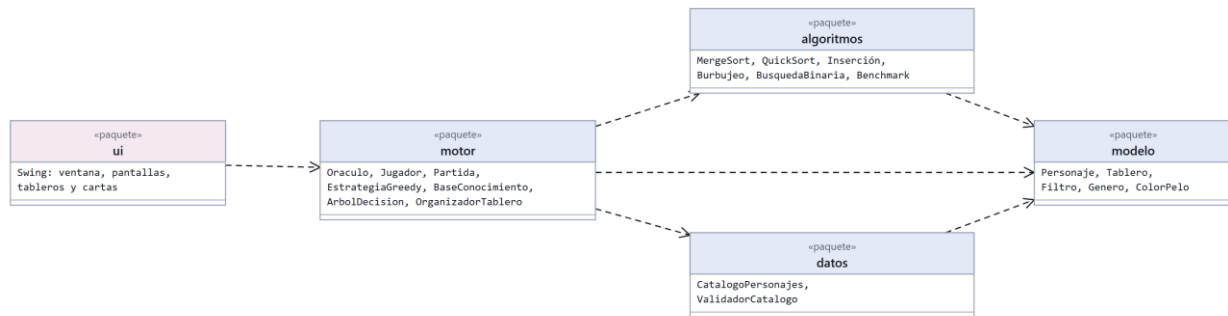


Figura 1. Paquetes del proyecto y sus dependencias.

Paquete	Responsabilidad	Clases principales
modelo	Las entidades del juego, sin lógica de partida.	Personaje, Tablero, Filtro, Genero, ColorPelo
datos	El catálogo de 23 personajes y sus precondiciones.	CatalogoPersonajes, ValidadorCatalogo
algoritmos	Ordenamientos, búsqueda binaria y medición de tiempos.	MergeSort, QuickSort, OrdenamientoInsercion, OrdenamientoBurbuja, BusquedaBinaria, Benchmark
motor	Reglas, turnos y estrategia de la máquina.	Oraculo, Jugador, JugadorHumano, JugadorMaquina, Partida, EstrategiaGreedy, BaseConocimiento, ArbolDecision, OrganizadorTablero
ui	Ventana Swing y pantallas.	VentanaPrincipal, PanelJugadorVsMaquina, PanelMaquinaVsMaquina, PanelOrdenamiento, PanelArbolDecision, TarjetaPersonaje
test	Pruebas y herramientas de análisis. No forman parte del juego.	PruebasAutomaticas, AnalisisOptimalidad, MedicionTiempos, GeneradorCapturas

2.2 Relaciones entre clases

Los dos diagramas de las páginas siguientes muestran las clases con sus atributos y métodos principales. El primero cubre el motor del juego; el segundo, los datos, los ordenamientos y el armado del tablero. En la notación, + es público, - privado y ~ de paquete. Las flechas son las de UML: triángulo para herencia (línea llena) y realización (línea punteada), rombo lleno para composición, rombo vacío para agregación, flecha simple para asociación y línea punteada para dependencia.

Diagrama de clases 1 · Motor del juego (paquetes motor y modelo)

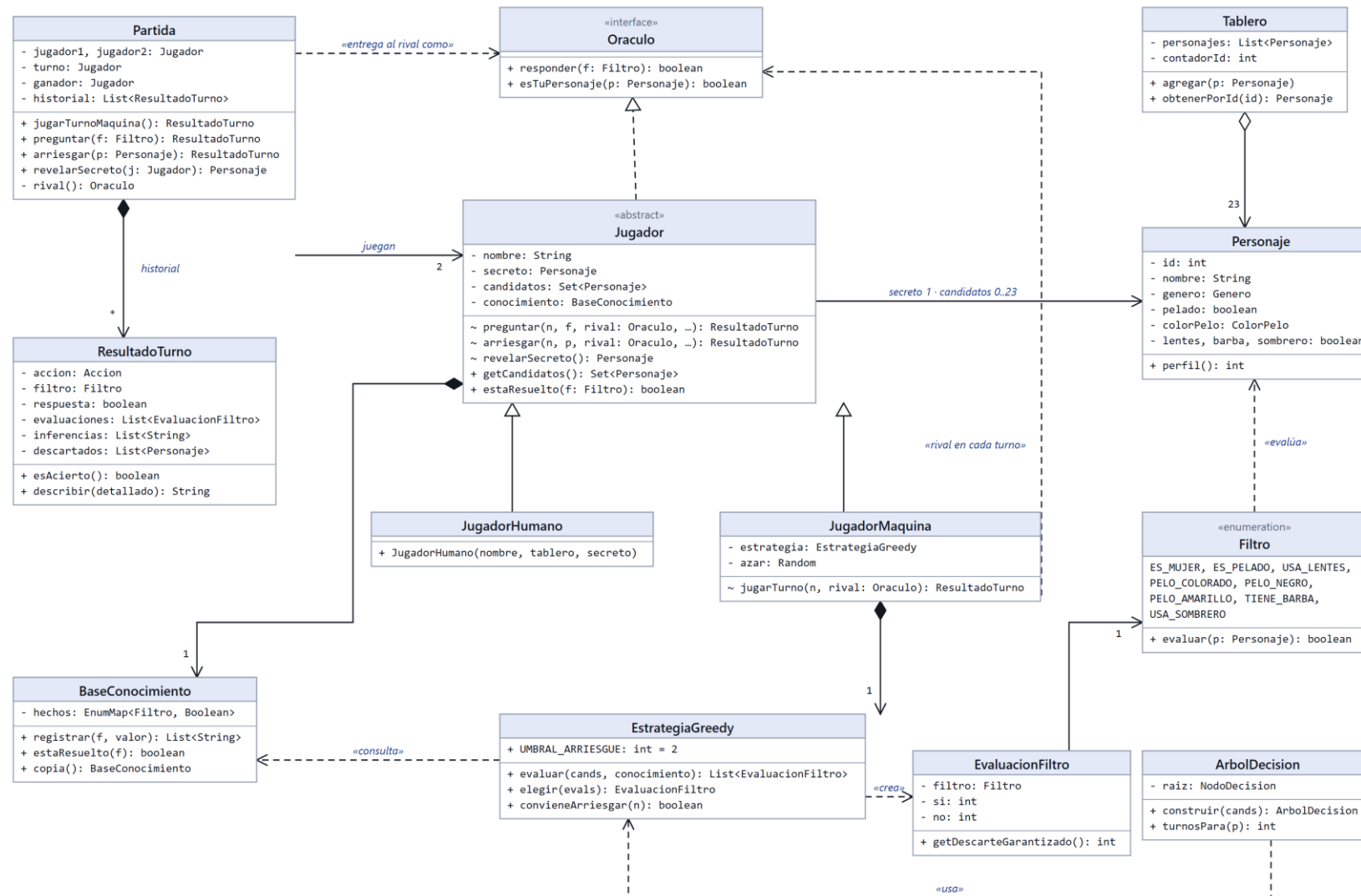


Figura 2. Diagrama de clases del motor del juego.

Diagrama de clases 2 · Datos, ordenamiento y armado del tablero

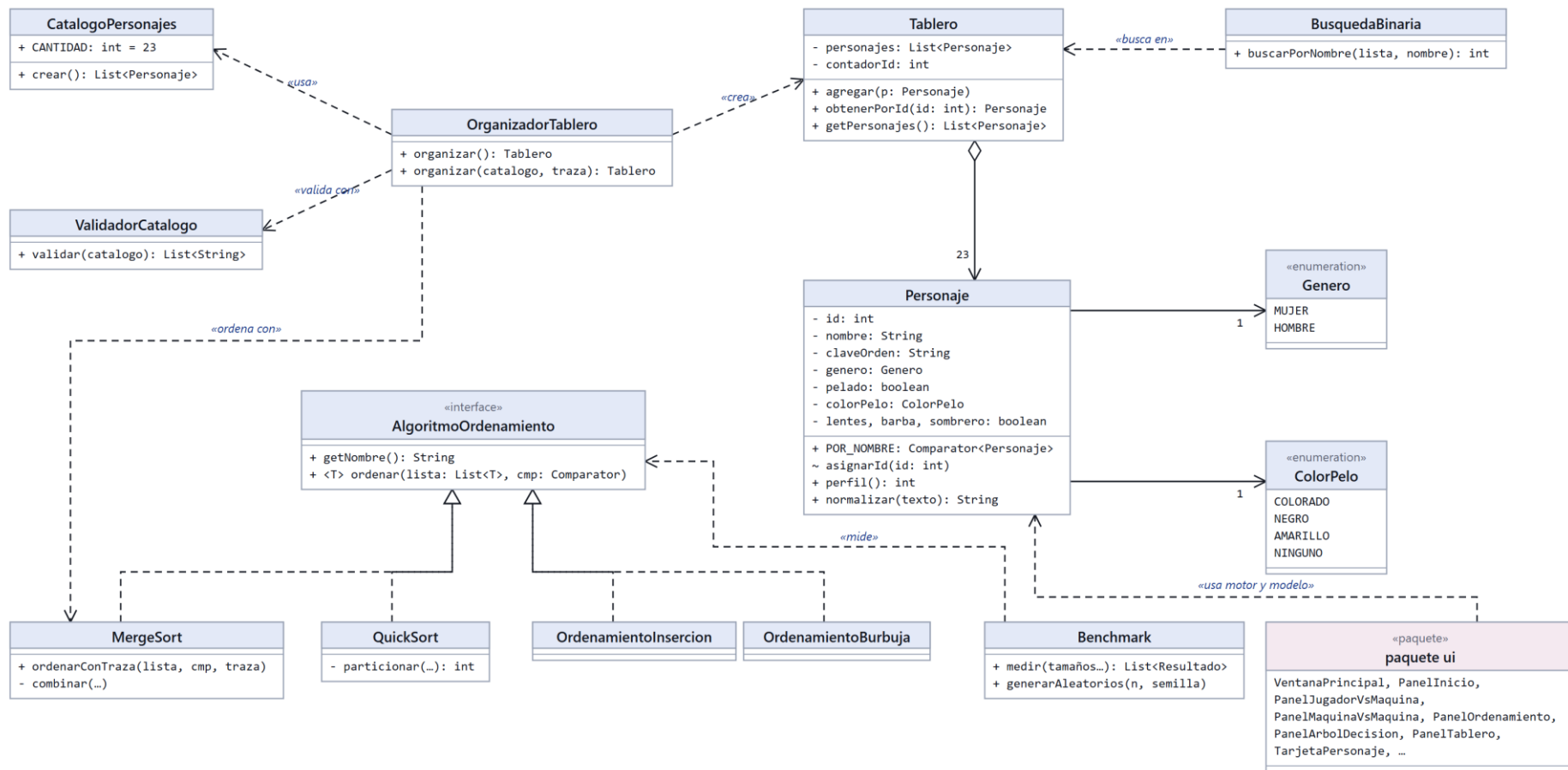


Figura 3. Diagrama de clases de datos, ordenamiento y armado del tablero.

Las relaciones más importantes y por qué están modeladas así:

Relación	Tipo	Por qué
Jugador → Oraculo	Realización	Cada jugador responde preguntas sobre su propio secreto. Es la única cara que ve el rival.
JugadorHumano, JugadorMaquina → Jugador	Herencia	Comparten el secreto, los candidatos, la base de conocimiento y la mecánica de preguntar y arriesgar. Sólo cambia quién decide: la interfaz o la estrategia.
Partida → Jugador (2)	Asociación	La partida conoce a los dos jugadores y controla de quién es el turno.
Partida ♦→ ResultadoTurno (*)	Composición	El historial pertenece a la partida y no existe sin ella.
Jugador ♦→ BaseConocimiento	Composición	Lo que un jugador sabe del rival es suyo. Hacia afuera se entrega sólo una copia.
JugadorMaquina ♦→ EstrategiaGreedy	Composición	La estrategia es parte de la máquina.
Jugador → Personaje	Asociación	Un secreto y un conjunto de 0 a 23 candidatos.
Tablero ◇→ Personaje (23)	Agregación	El tablero ordena y numera personajes que se crean en el catálogo.
JugadorMaquina ↔ Oraculo	Dependencia	La máquina recibe al rival como parámetro en cada turno y no lo guarda en ningún campo. Esa es la garantía de que no accede al secreto.
OrganizadorTablero ↔ ValidadorCatalogo, MergeSort, Tablero	Dependencia	Los usa para armar el tablero y no guarda referencias.
MergeSort, QuickSort, Inserción, Burbujeo → AlgoritmoOrdenamiento	Realización	El Benchmark mide los cuatro con el mismo código, por polimorfismo.

En el diagrama se ven dos principios SOLID de la materia. **Inversión de dependencias:** la máquina depende de la abstracción **Oraculo**, no del jugador concreto. **Abierto/cerrado:** para agregar un ordenamiento basta con implementar **AlgoritmoOrdenamiento**, sin tocar **Benchmark**.

2.3 Justificación del modelo de datos

Estructura	Dónde	Por qué	Costo
List<Personaje> (ArrayList)	Tablero	Hace falta orden (alfabético) y acceso por posición: el personaje con ID i está en la posición i – 1.	Por ID O(1); agregar al final O(1) amortizado; búsqueda binaria O(log n).
Arreglo auxiliar T[]	MergeSort	La combinación necesita un arreglo del mismo tamaño para intercalar.	Θ(n) de memoria.
Set<Personaje> (LinkedHashSet)	Jugador.candidatos	Los candidatos son un conjunto: no hay repetidos y se pregunta pertenencia. LinkedHashSet conserva el orden del tablero, así las trazas son reproducibles.	contains y remove O(1); recorrido O(c).
Map<Filtro, Boolean> (EnumMap)	BaseConocimiento	Asocia cada filtro resuelto con su valor. EnumMap es un arreglo indexado por el ordinal del enum: más compacto y rápido que un HashMap.	get, put y containsKey O(1).
Set<Filtro> (EnumSet)	BaseConocimiento	El grupo exclusivo {pelado, colorado, negro, amarillo}. Internamente es una máscara de bits.	O(1).
HashSet<String>, HashMap<Integer, Personaje>	ValidadorCatalogo	Detectar nombres y perfiles repetidos en una sola pasada.	O(1) por personaje, O(n) en total.
List<ResultadoTurno>	Partida	Registro cronológico de la partida.	Agregar O(1).

Estructura	Dónde	Por qué	Costo
enum Filtro	modelo	Los filtros son un conjunto cerrado. Cada constante implementa evaluar(Personaje), así que agregar uno no obliga a tocar ningún switch.	evaluar O(1).

Además, `Personaje.perfil()` codifica los 8 filtros de un personaje en un entero de 8 bits. Dos personajes con el mismo perfil serían indistinguibles. El validador usa ese número como clave de un `HashMap` para detectarlos, y la herramienta de análisis lo usa para representar conjuntos de candidatos como máscaras de bits.

3. Bitácora de desarrollo

3.1 Etapas

Fecha	Etapas	Qué se hizo
26/08/2026	Primer borrador	Proyecto en IntelliJ con las clases Personajes y Acciones: generación aleatoria de personajes con muchos atributos (profesión, mascota, música). No llegó a compilar y se descartó, porque la consigna pide personajes fijos y filtros definidos.
29/08/2026	Versión 1	Adivina Quién por consola. El jugador escribía preguntas en lenguaje natural y un filtro de palabras clave las traducía a un atributo. Sólo adivinaba el humano: la máquina no jugaba.
26/09/2026	Versión 2 (la que se entrega)	Con la documentación formal del TP rehicimos el proyecto desde cero, en este orden: modelo y catálogo; validaciones; MergeSort y los ordenamientos de comparación; motor de turnos con Oráculo; estrategia Greedy y base de conocimiento; árbol de decisión; pruebas automáticas; interfaz Swing; benchmark y análisis de optimalidad; ajuste del criterio de parada; este documento.
[COMPLETAR]	[COMPLETAR]	[COMPLETAR: revisión del equipo, pruebas, ajustes posteriores]

La versión 2 se desarrolló en una jornada de trabajo intensivo con asistencia de IA (ver 3.3).

3.2 Herramientas

- **Java 17**, compilado con el JDK 25 y la opción `--release 17`, para que corra en cualquier JDK desde el 17. No usa librerías externas.
- **IntelliJ IDEA 2025.3** como entorno de desarrollo.
- **Swing** para la interfaz. Las caras de los personajes se dibujan con Graphics2D a partir de sus atributos, sin imágenes.
- **Git y GitHub** para versionar el proyecto.
- **Claude Code** (Anthropic) como asistente de IA; ver 3.3.
- **Microsoft Word** para este documento y **Google Chrome** para exportar los diagramas a imagen.

3.3 Uso de inteligencia artificial

Sí, usamos IA. Trabajamos con Claude (modelo Claude Opus 5.5, de Anthropic) a través de Claude Code, un asistente que lee y escribe los archivos del proyecto y ejecuta comandos en la computadora.

- **Para qué.** A partir de la consigna, la IA propuso el diseño, escribió el código Java, las pruebas y las herramientas de análisis, generó los diagramas UML y las capturas, y redactó el borrador de este documento.
- **Qué decidió el equipo.** Rehacer el proyecto desde cero en lugar de continuar la versión 1; usar una interfaz Swing; agregar atributos para que los 23 personajes sean distinguibles; decir "pelado" en lugar de "calvo"; entregar el documento en Word. [COMPLETAR: otras decisiones y revisiones del equipo.]

- **Cómo se verificó.** Todo el código compila y pasa 54 pruebas automáticas. Los tiempos, las cantidades de turnos y los contraejemplos de este documento salen de ejecutar el programa, no de estimaciones.
- **Lo que no reemplaza.** En la defensa, cada integrante tiene que poder explicar cualquier parte del código.
[COMPLETAR: cómo se repartió el estudio del código.]

3.4 División del trabajo

Integrante	Tareas
[COMPLETAR]	[COMPLETAR]
[COMPLETAR]	[COMPLETAR]
[COMPLETAR]	[COMPLETAR]

3.5 Problemas encontrados y cómo se resolvieron

Problema	Solución
Con los filtros de la consigna, 23 personajes no entran en 16 perfiles: 7 quedaban idénticos a otro.	Agregar barba y sombrero (48 perfiles), y una precondition que rechaza el catálogo si dos personajes tienen el mismo perfil.
La versión 1 no cumplía la consigna: no tenía Máquina vs Máquina, ni ordenamiento, ni estrategia.	Reescritura completa. Sólo se conservó la idea de un tablero de atributos.
Encapsular el secreto sin perder la posibilidad de mostrarlo al final.	El secreto es private en Jugador y no tiene getter; hacia afuera, el jugador sólo es un Oraculo. Partida.revelarSecreto() funciona únicamente cuando la partida terminó.
Un ordenamiento de 23 elementos tarda alrededor de un microsegundo: menos de lo que el reloj mide con precisión.	Repetir cada ordenamiento 20.000 veces sobre copias preparadas de antemano, calentar el JIT antes de medir y reportar la mediana de 7 corridas.
Para probar que la máquina siempre adivina hacía falta que el rival no le ganara antes.	Una prueba que hace jugar a la máquina sola contra cada uno de los 23 secretos.
Empates en la función de selección: en el primer turno, ¿Es mujer? y ¿Usa lentes? descartan 11 cada uno.	Desempate determinista por el orden del enum. La máquina es predecible y el árbol de decisión es el mismo en cada ejecución.
El análisis de optimalidad mostró que, con 3 candidatos, la máquina preguntaba cuando le convenía arriesgar.	El umbral de parada pasó de 2 a 3 candidatos. Con ese cambio, en el tablero completo la estrategia empató al óptimo exacto (sección 5.4).
Las capturas de la interfaz salían con textos cortados al dibujarlas en alta resolución.	Generarlas con la escala de interfaz de Java en 2 (-Dsun.java2d.uiScale=2), para que los textos midan lo mismo al armar la pantalla y al dibujarla.

3.6 Fragmentos de código y su complejidad

Estas son capturas del código real, con sus números de línea. En la esquina de cada una va su complejidad temporal. La notación es: n , la cantidad de personajes del tablero (23); c , los candidatos que le quedan al jugador ($c \leq n$); f , la cantidad de filtros (8); y h , la altura del árbol de decisión.

MergeSort.java · MergeSort: dividir, conquistar y combinar

 $\Theta(n \log n)$

```

47 private <T> void ordenarRango(T[] datos, T[] auxiliar, int inicio, int fin,
48                               Comparator<? super T> comparador, Consumer<String> traza, int nivel) {
49     if (inicio >= fin) {
50         return; // caso base: 0 o 1 elemento ya está ordenado
51     }
52     int medio = (inicio + fin) >> 1;
53     if (traza != null) {
54         traza.accept(sangria(nivel) + "dividir [" + inicio + ".." + fin + "] en ["
55                     + inicio + ".." + medio + "] y [" + (medio + 1) + ".." + fin + "]);
56     }
57     ordenarRango(datos, auxiliar, inicio, medio, comparador, traza, nivel + 1);
58     ordenarRango(datos, auxiliar, medio + 1, fin, comparador, traza, nivel + 1);
59     combinar(datos, auxiliar, inicio, medio, fin, comparador);
60     if (traza != null) {
61         traza.accept(sangria(nivel) + "combinar [" + inicio + ".." + fin + "] -> "
62                     + Arrays.toString(Arrays.copyOfRange(datos, inicio, fin + 1)));
63     }
64 }
65
66 /** Intercala datos[inicio..medio] y datos[medio+1..fin], que ya están ordenados. O(fin - inicio). */
67 private <T> void combinar(T[] datos, T[] auxiliar, int inicio, int medio, int fin,
68                           Comparator<? super T> comparador) {
69     System.arraycopy(datos, inicio, auxiliar, inicio, fin - inicio + 1);
70     int izq = inicio;
71     int der = medio + 1;
72     for (int k = inicio; k <= fin; k++) {
73         if (izq > medio) {
74             datos[k] = auxiliar[der++];
75         } else if (der > fin) {
76             datos[k] = auxiliar[izq++];
77         } else if (comparador.compare(auxiliar[der], auxiliar[izq]) < 0) {
78             datos[k] = auxiliar[der++]; // sólo si es estrictamente menor: así es estable
79         } else {
80             datos[k] = auxiliar[izq++];
81         }
82     }
83 }

```

Figura 4. MergeSort: dividir, conquistar y combinar (MergeSort.java). Complejidad: $\Theta(n \log n)$.

MergeSort. Cada nivel de la recursión combina n elementos en total y hay $\lceil \log_2 n \rceil$ niveles. La recurrencia es $T(n) = 2T(n/2) + \Theta(n)$, que por el caso 2 del teorema maestro ($a = 2$, $b = 2$, $f(n) = n = n^{(\log_b a)}$) da $\Theta(n \log n)$. La comparación estricta en **combinar** hace que el algoritmo sea estable.

Tablero.java · ID autoincremental al agregar

 $O(1)$

```

18 /**
19  * Agrega un personaje al final y le asigna el siguiente ID.
20  * Precondición: el personaje no puede ir antes que el último (La lista ya tiene que llegar ordenada).
21  */
22 public void agregar(Personaje p) {
23     if (!personajes.isEmpty()) {
24         Personaje ultimo = personajes.get(personajes.size() - 1);
25         if (Personaje.POR_NOMBRE.compare(ultimo, p) > 0) {
26             throw new IllegalArgumentException(p + " rompe el orden del tablero (va antes que " + ultimo + ")");
27         }
28     }
29     contadorId++;
30     p.asignarId(contadorId);
31     personajes.add(p);
32 }

```

Figura 5. ID autoincremental al agregar (Tablero.java). Complejidad: $O(1)$.

Tablero.agregar. Compara con el último personaje y agrega al final: $O(1)$. El contador asigna el ID autoincremental. La verificación de orden hace que la lista quede ordenada por construcción.

OrganizadorTablero.java · La máquina arma el tablero

 $\Theta(n \log n)$

```

28 public static Tablero organizar(List<Personaje> catalogo, Consumer<String> traza) {
29     List<String> errores = ValidadorCatalogo.validar(catalogo);
30     if (!errores.isEmpty()) {
31         throw new IllegalArgumentException("Catálogo inválido:\n" + String.join("\n", errores));
32     }
33     List<Personaje> ordenados = new ArrayList<>(catalogo);
34     new MergeSort().ordenarConTraza(ordenados, Personaje.POR_NOMBRE, traza);
35
36     Tablero tablero = new Tablero();
37     for (Personaje p : ordenados) {
38         tablero.agregar(p); // ID = 1, 2, 3, ... en el orden en que se agregan
39         if (traza != null) {
40             traza.accept("agregar " + p.getNombre() + " -> ID " + p.getId());
41         }
42     }
43     return tablero;
44 }

```

Figura 6. La máquina arma el tablero (OrganizadorTablero.java). Complejidad: $\Theta(n \log n)$.

OrganizadorTablero.organizar. Validar es $O(n)$, ordenar $\Theta(n \log n)$ y agregar $O(n)$. En total, $\Theta(n \log n)$.

BusquedaBinaria.java · Búsqueda binaria recursiva del arriesgue

 $O(\log n)$

```

23 private static int buscar(List<Personaje> ordenada, String clave, int inicio, int fin) {
24     if (inicio > fin) {
25         return -1; // rango vacío: no está
26     }
27     int medio = (inicio + fin) >> 1;
28     int comparacion = clave.compareTo(ordenada.get(medio).getClaveOrden());
29     if (comparacion == 0) {
30         return medio;
31     }
32     return comparacion < 0
33         ? buscar(ordenada, clave, inicio, medio - 1)
34         : buscar(ordenada, clave, medio + 1, fin);
35 }

```

Figura 7. Búsqueda binaria recursiva del arriesgue (BusquedaBinaria.java). Complejidad: $O(\log n)$.

Búsqueda binaria. $T(n) = T(n/2) + O(1)$, es decir, $O(\log n)$. Con 23 personajes hace a lo sumo 5 comparaciones.

EstrategiaGreedy.java · Función de evaluación y de selección

 $O(f \cdot c)$

```

31  /** Función de evaluación: cuenta SÍ y NO de cada filtro no resuelto.  $O(f \cdot c)$ . */
32  public List<EvaluacionFiltro> evaluar(Collection<Personaje> candidatos, BaseConocimiento conocimiento) {
33      List<EvaluacionFiltro> evaluaciones = new ArrayList<>();
34      for (Filtro filtro : Filtro.values()) {
35          if (conocimiento.estaResuelto(filtro)) {
36              continue; // La dependencia lógica ya lo resolvió: no se evalúa
37          }
38          int si = 0;
39          for (Personaje p : candidatos) {
40              if (filtro.evaluar(p)) {
41                  si++;
42              }
43          }
44          evaluaciones.add(new EvaluacionFiltro(filtro, si, candidatos.size() - si));
45      }
46      return evaluaciones;
47  }
48
49  /**
50   * Función de selección: el máximo descarte garantizado. En empate gana el primero en el orden
51   * del enum (la comparación es estricta). Devuelve null si ningún filtro separa candidatos.  $O(f)$ .
52   */
53  public EvaluacionFiltro elegir(List<EvaluacionFiltro> evaluaciones) {
54      EvaluacionFiltro mejor = null;
55      for (EvaluacionFiltro e : evaluaciones) {
56          if (e.esUtil() && (mejor == null || e.getDescarteGarantizado() > mejor.getDescarteGarantizado())) {
57              mejor = e;
58          }
59      }
60      return mejor;
61  }
62
63  /** Criterio de parada: con 1 candidato se gana seguro; con 2 o 3, arriesgar cuesta menos turnos esperados. */
64  public boolean convieneArriesgar(int candidatos) {
65      return candidatos <= UMBRAL_ARRIESGAR;
66  }

```

Figura 8. Función de evaluación y de selección (EstrategiaGreedy.java). Complejidad: $O(f \cdot c)$.

Evaluación y selección del Greedy. evaluar recorre f filtros $\times$ c candidatos: $O(f \cdot c)$. elegir recorre las evaluaciones: $O(f)$. Como f está fijo en 8, un turno es lineal en la cantidad de candidatos.

JugadorMaquina.java · Decisión de la máquina en cada turno

 $O(f \cdot c)$

```

34  /** Un turno completo: decide si arriesga o qué pregunta, y lo ejecuta.  $O(f \cdot c)$ . */
35  ResultadoTurno jugarTurno(int numeroTurno, Oraculo rival) {
36      Set<Personaje> candidatos = getCandidatos();
37      int n = candidatos.size();
38
39      if (estrategia.convieneArriesgar(n)) {
40          Personaje elegido = elegirAlAzar(candidatos);
41          String motivo;
42          if (n == 1) {
43              motivo = "queda un solo candidato: arriesgar es ganar";
44          } else if (n == 2) {
45              motivo = "quedan 2 candidatos: arriesgar de a uno cuesta 1,5 turnos esperados; preguntar antes, 2";
46          } else {
47              motivo = "quedan 3 candidatos: arriesgar de a uno cuesta 2 turnos esperados; preguntar antes, 2,33";
48          }
49          return arriesgar(numeroTurno, elegido, rival, null, motivo);
50      }
51
52      List<EvaluacionFiltro> evaluaciones = estrategia.evaluar(candidatos, conocimientoInterno());
53      EvaluacionFiltro mejor = estrategia.elegir(evaluaciones);
54      if (mejor == null) {
55          // No debería pasar con un catálogo válido (perfiles únicos), pero si pasa no hay nada que preguntar.
56          return arriesgar(numeroTurno, elegirAlAzar(candidatos), rival, evaluaciones,
57              "ningún filtro separa a los candidatos: sólo queda arriesgar");
58      }
59      String motivo = mejor.getFiltro().getPregunta() + " descarta al menos " + mejor.getDescarteGarantizado()
60          + " de " + n + " candidatos, conteste lo que conteste";
61      return preguntar(numeroTurno, mejor.getFiltro(), rival, evaluaciones, motivo);
62  }

```

Figura 9. Decisión de la máquina en cada turno (JugadorMaquina.java). Complejidad: $O(f \cdot c)$.

Turno de la máquina. Decide entre arriesgar ($O(c)$ para elegir) y preguntar ($O(f \cdot c)$ para evaluar, más $O(c)$ para descartar). Nunca recibe ni guarda el secreto del rival: sólo el Oraculo, como parámetro.

Jugador.java · Descarte de candidatos

 $O(c)$

```

52  /** Hace una pregunta al rival y descarta los candidatos que no coinciden con la respuesta.  $O(c)$ . */
53  ResultadoTurno preguntar(int numeroTurno, Filtro filtro, Oraculo rival,
54      List<EvaluacionFiltro> evaluaciones, String motivo) {
55      if (conocimiento.estaResuelto(filtro)) {
56          throw new IllegalArgumentException(filtro.getPregunta() + " ya está resuelta");
57      }
58      int antes = candidatos.size();
59      boolean respuesta = rival.responder(filtro);
60      List<String> inferencias = conocimiento.registrar(filtro, respuesta);
61
62      List<Personaje> descartados = new ArrayList<>();
63      Iterator<Personaje> it = candidatos.iterator();
64      while (it.hasNext()) {
65          Personaje p = it.next();
66          if (filtro.evaluar(p) != respuesta) {
67              descartados.add(p);
68              it.remove();
69          }
70      }
71      verificarInvariante();
72      return new ResultadoTurno(numeroTurno, nombre, ResultadoTurno.Accion.PREGUNTA, filtro, respuesta, null,
73          evaluaciones, inferencias, descartados, antes, candidatos.size(), motivo);
74  }

```

Figura 10. Descarte de candidatos (Jugador.java). Complejidad: $O(c)$.

Descarte de candidatos. Un recorrido del conjunto con un iterador que elimina en $O(1)$: $O(c)$ en total. Registrar la respuesta en la base de conocimiento es $O(1)$.

BaseConocimiento.java · Dependencia lógica entre filtros

O(1)

```

55 private boolean aplicarReglas(List<String> inferencias) {
56     boolean cambio = false;
57
58     // Regla 1: grupo exclusivo del pelo.
59     Filtro verdadero = null;
60     int falsos = 0;
61     Filtro pendiente = null;
62     for (Filtro f : PELO) {
63         Boolean v = hechos.get(f);
64         if (Boolean.TRUE.equals(v)) {
65             verdadero = f;
66         } else if (Boolean.FALSE.equals(v)) {
67             falsos++;
68         } else {
69             pendiente = f;
70         }
71     }
72     if (verdadero != null) {
73         for (Filtro f : PELO) {
74             if (!hechos.containsKey(f)) {
75                 cambio |= inferir(f, false, "porque " + verdadero.getPregunta() + " es SÍ", inferencias);
76             }
77         }
78     } else if (falsos == PELO.size() - 1 && pendiente != null) {
79         cambio |= inferir(pendiente, true, "porque las otras tres opciones de pelo son NO", inferencias);
80     }
81
82     // Regla 2: mujer => sin barba; barba => no es mujer.
83     if (Boolean.TRUE.equals(hechos.get(Filtro.ES_MUJER)) && !hechos.containsKey(Filtro.TIENE_BARBA)) {
84         cambio |= inferir(Filtro.TIENE_BARBA, false, "porque es mujer", inferencias);
85     }
86     if (Boolean.TRUE.equals(hechos.get(Filtro.TIENE_BARBA)) && !hechos.containsKey(Filtro.ES_MUJER)) {
87         cambio |= inferir(Filtro.ES_MUJER, false, "porque tiene barba", inferencias);
88     }
89     return cambio;
90 }

```

Figura 11. Dependencia lógica entre filtros (BaseConocimiento.java). Complejidad: $O(1)$.

Dependencia lógica. Revisa el grupo exclusivo del pelo (4 filtros) y la regla de la barba. Con 8 filtros en total, es $O(1)$.

ArbolDecision.java · Árbol de decisión por Divide y Conquista

 $O(f \cdot n \cdot h)$

```

30 private static NodoDecision construirNodo(List<Personaje> candidatos, BaseConocimiento conocimiento,
31                                           EstrategiaGreedy estrategia) {
32     if (estrategia.convieneArriesgar(candidatos.size())) {
33         return NodoDecision.hoja(candidatos); // caso base
34     }
35     EvaluacionFiltro mejor = estrategia.elegir(estrategia.evaluar(candidatos, conocimiento));
36     if (mejor == null) {
37         return NodoDecision.hoja(candidatos);
38     }
39     Filtro filtro = mejor.getFiltro();
40
41     // Dividir
42     List<Personaje> grupoSi = new ArrayList<>();
43     List<Personaje> grupoNo = new ArrayList<>();
44     for (Personaje p : candidatos) {
45         (filtro.evaluar(p) ? grupoSi : grupoNo).add(p);
46     }
47     BaseConocimiento sabeSi = conocimiento.copia();
48     sabeSi.registrar(filtro, true);
49     BaseConocimiento sabeNo = conocimiento.copia();
50     sabeNo.registrar(filtro, false);
51
52     // Conquistar y combinar
53     return new NodoDecision(filtro, candidatos,
54                             construirNodo(grupoSi, sabeSi, estrategia),
55                             construirNodo(grupoNo, sabeNo, estrategia));
56 }

```

Figura 12. Árbol de decisión por Divide y Conquista (ArbolDecision.java). Complejidad: $O(f \cdot n \cdot h)$.

Árbol de decisión. En cada nivel del árbol, entre todos los nodos, se evalúan los f filtros sobre los n candidatos: $O(f \cdot n)$ por nivel, $O(f \cdot n \cdot h)$ en total. Con divisiones parejas, $h \approx \log_2 n$.

Jugador.java · El secreto sólo se consulta, nunca se lee

 $O(1)$

```

37 @Override
38 public final boolean responder(Filtro filtro) {
39     return filtro.evaluar(secreto);
40 }
41
42 @Override
43 public final boolean esTuPersonaje(Personaje candidato) {
44     return candidato == secreto;
45 }

```

Figura 13. El secreto sólo se consulta, nunca se lee (Jugador.java). Complejidad: $O(1)$.

El secreto. Responder una pregunta o verificar un arriesgue es $O(1)$. Los métodos son final: ninguna subclase puede cambiar cómo se responde.

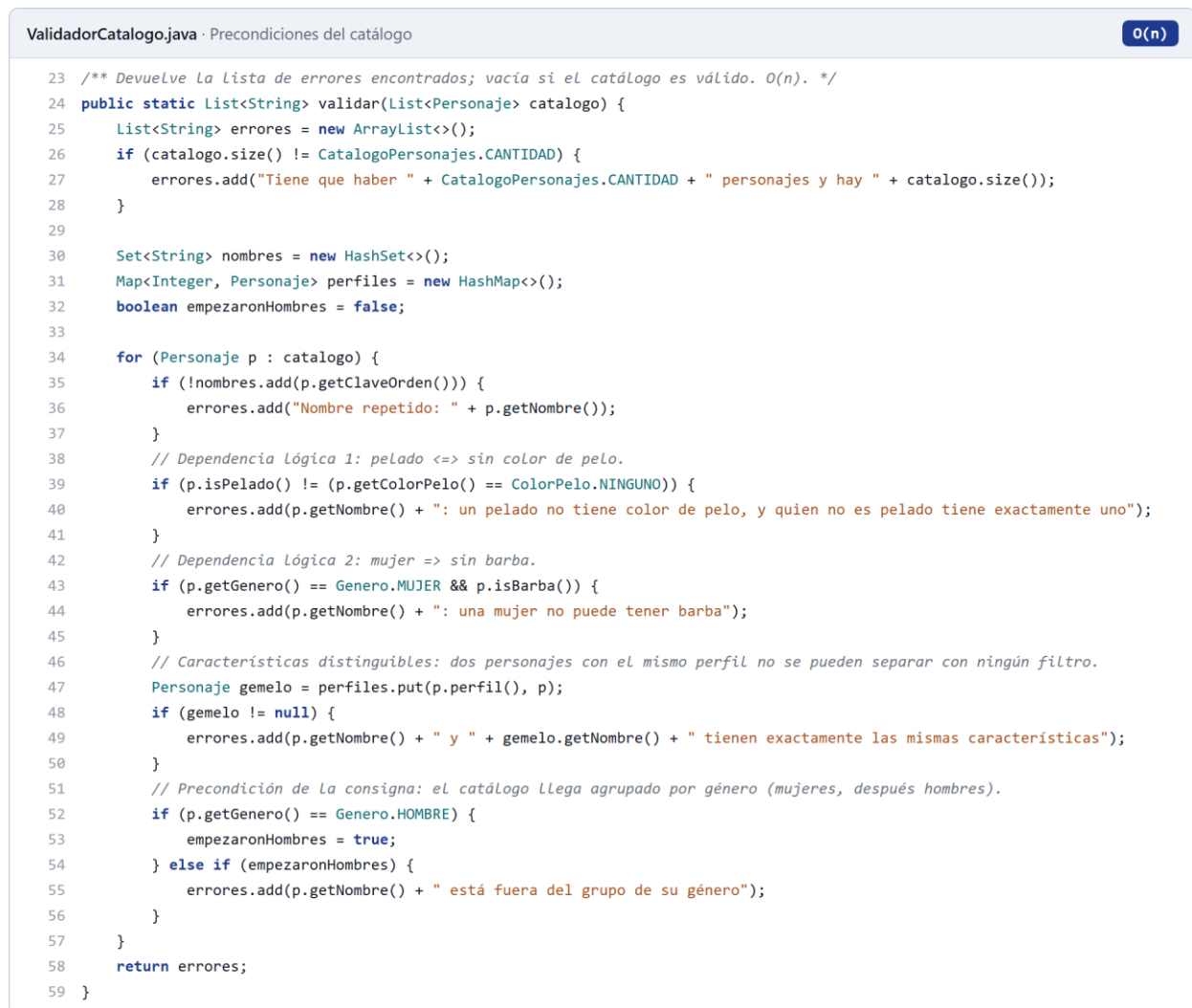


Figura 14. Precondiciones del catálogo (ValidadorCatalogo.java). Complejidad: $O(n)$.

Precondiciones del catálogo. Una sola pasada con HashSet y HashMap: $O(n)$.

4. Justificación algorítmica de Divide y Conquista

4.1 Algoritmo de ordenamiento inicial

El catálogo llega como pide la consigna: primero las 11 mujeres, después los 12 hombres, sin ningún otro orden dentro de cada grupo. La máquina lo ordena con MergeSort, que aplica los tres pasos de Divide y Conquista:

- **Dividir:** parte el rango [inicio..fin] por la mitad.
- **Conquistar:** ordena cada mitad recursivamente. El caso base es un rango de 0 o 1 elemento, que ya está ordenado.
- **Combinar:** intercala las dos mitades ordenadas en un solo recorrido, en $O(n)$.

$$T(n) = 2 \cdot T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$$

Por el teorema maestro, $a = 2$, $b = 2$ y $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$: es el caso 2, y el costo es $\Theta(n \log n)$ en el mejor caso, en el promedio y en el peor. Con $n = 23$, MergeSort hace 22 divisiones, 22 combinaciones y 76 comparaciones. La pantalla "Ordenamiento del tablero" muestra la traza completa.

Después de ordenar, la máquina agrega los personajes al **Tablero** de a uno. **Tablero.agregar()** les asigna el ID con un contador que se incrementa en cada agregado: Ana recibe el 1, Bruno el 2, y así hasta Valentina, que recibe el 23. El tablero verifica además que cada personaje nuevo no vaya antes que el último. La lista queda ordenada por construcción, y un agregado fuera de orden lanza una excepción.

4.2 Elección entre MergeSort y QuickSort

Criterio	MergeSort	QuickSort
Peor caso	$\Theta(n \log n)$	$O(n^2)$, si el pivote cae siempre en un extremo
Estabilidad	Estable	No estable
Memoria extra	$\Theta(n)$	$O(\log n)$ de pila en promedio
Comparaciones con $n = 23$	76	118
Tiempo con $n = 23$ (mediana)	0,00110 ms	0,00071 ms

Elegimos MergeSort por tres motivos:

- **Garantiza $\Theta(n \log n)$ aun en el peor caso.** No depende de la elección del pivote ni de cómo venga la entrada.
- **Es estable.** Dos personajes con la misma clave conservan su orden relativo. Si el criterio cambiara (por ejemplo, ordenar por color de pelo), los empates seguirían agrupados por género, como en el catálogo.
- **Su traza se sigue fácil en pantalla,** porque las divisiones siempre son por la mitad.

Su desventaja, la memoria extra $\Theta(n)$, no importa con 23 personajes. QuickSort quedó implementado sólo para la comparación de tiempos.

4.3 Criterio de ordenamiento

Ordenamos por nombre, en orden alfabético, sin distinguir mayúsculas ni tildes. Cada personaje calcula una sola vez su clave de ordenamiento: `Personaje.normalizar` descompone el texto (forma NFD), borra las marcas diacríticas y lo pasa a minúsculas. Así, "Lucía" y "lucia" comparan igual, y cada comparación es una comparación de cadenas común.

El criterio no es arbitrario: hace que el arriesgue también sea Divide y Conquista. Cuando el jugador escribe un nombre, `BusquedaBinaria` lo busca en el tablero ordenado en $O(\log n)$: con 23 personajes, 5 comparaciones como máximo. En un tablero ordenado sólo por género habría que buscar de forma secuencial, en $O(n)$.

4.4 Estrategia de selección del filtro

En cada turno, la máquina tiene un conjunto de candidatos C (los personajes que todavía pueden ser el secreto del rival) y un conjunto de filtros sin resolver. Cada filtro divide a C en dos grupos: los que responderían SÍ y los que responderían NO. La máquina no sabe con qué grupo se va a quedar, porque la respuesta depende del rival. Lo que sí sabe es que, conteste lo que conteste, descarta por lo menos al grupo más chico. La estrategia elige el filtro que maximiza ese **descarte garantizado**.

Es un criterio **minimax**: minimiza el grupo que puede quedar en el peor caso, $\max(\text{SÍ}, \text{NO})$, lo que equivale a maximizar $\min(\text{SÍ}, \text{NO})$. El filtro ideal parte a los candidatos por la mitad, como la búsqueda binaria parte un arreglo.

4.5 Función de evaluación del filtro

$$\text{evaluación}(f, C) = \min(|\{p \in C : f(p) = \text{SÍ}\}| , |\{p \in C : f(p) = \text{NO}\}|)$$

`EstrategiaGreedy.evaluar()` recorre los filtros sin resolver y, para cada uno, cuenta los candidatos que cumplen: $O(f \cdot c)$. `EstrategiaGreedy.elegir()` se queda con el de mayor evaluación. Si hay empate, gana el primero en el orden del enum. Un filtro con evaluación 0 no separa a nadie (todos los candidatos contestarían lo mismo) y nunca se elige. Así evalúa la máquina su primer turno:

Filtro	SÍ	NO	Descarte garantizado
¿Es mujer?	11	12	11 ← elegido
¿Es pelado?	4	19	4
¿Usa lentes?	11	12	11
¿Tiene pelo colorado?	6	17	6

Filtro	SÍ	NO	Descarte garantizado
¿Tiene pelo negro?	7	16	7
¿Tiene pelo amarillo?	6	17	6
¿Tiene barba?	6	17	6
¿Usa sombrero?	8	15	8

¿Es mujer? y ¿Usa lentes? empatan en 11. Gana ¿Es mujer? porque está primero en el enum.

4.6 Relación con el árbol de decisión del juego

Cada partida de la máquina es un recorrido por un árbol binario de decisión. Los nodos internos son preguntas, las ramas son las respuestas SÍ y NO, y las hojas son los arriesgues. La clase `ArbolDecision` construye ese árbol de antemano, con la misma estrategia que usa el juego, y lo hace por Divide y Conquista:

- **Dividir:** elige el mejor filtro para los candidatos y los separa en dos grupos.
- **Conquistar:** construye recursivamente el subárbol de cada grupo.
- **Combinar:** une los dos subárboles en un nodo.

Una de las pruebas automáticas verifica que la máquina, en partida, hace exactamente las preguntas del árbol.

Para los 23 personajes, el árbol tiene altura 4: la máquina nunca necesita más de 4 preguntas. Contando los arriesgues finales, adivina en 6 turnos en el peor caso y en 5,04 en promedio, si los 23 secretos son igual de probables. La pantalla "Árbol de decisión" lo muestra completo (figura siguiente).

Árbol de decisión de la máquina
Altura: 4 preguntas · peor caso: 6 turnos · promedio: 5,04 turnos (contando el arriesgue final)

[→ Menú](#)

ÁRBOL COMPLETO (23 PERSONAJES)

CÓMO LEERLO

Cada nodo es la pregunta que elegiría el Greedy con esos candidatos: la de mayor **descarte garantizado**, $\min(SÍ, NO)$. La construcción es Divide y Conquista: el filtro divide a los candidatos, se arma el subárbol de cada grupo y el nodo lo combina. Las hojas son arriesgues: con 1 candidato se gana 2 o 3, se arriesga de a uno, en orden, hasta acertar.

TURNOS QUE NECESITA LA MÁQUINA POR SECRETO

ID	Secreto	Turnos
1	Ana	5
2	Bruno	4
3	Carla	4
4	Diego	4
5	Elena	6
6	Federico	5
7	Florencia	4
8	Gustavo	4
9	Hernán	5
10	Joaquín	5
11	Julietta	5
12	Lucas	5
13	Lucía	5
14	Martín	6
15	Martina	6
16	Micaela	4
17	Nicolás	5
18	Paula	5
19	Ramiro	6
20	Sergio	6
21	Sofía	6
22	Tomás	6
23	Valentina	5

Figura 15. Pantalla "Árbol de decisión": el árbol que genera el Greedy y los turnos que necesita la máquina para cada secreto.

La relación con la búsqueda binaria es directa. Si cada pregunta partiera a los candidatos exactamente por la mitad, harían falta $\lceil \log_2 23 \rceil = 5$ preguntas para aislar a cualquier personaje, y ningún método puede hacerlo con menos: es la cota inferior de cualquier árbol binario con 23 hojas. Nuestros filtros no siempre parten por la mitad, así que la estrategia se acerca a ese ideal sin alcanzarlo siempre. Por eso conviene arriesgar al final en lugar de seguir preguntando (sección 4.8).

4.7 Cómo se descartan los candidatos en cada turno

`Jugador.preguntar()` recibe la respuesta del Oráculo, recorre los candidatos y elimina a los que contestarían distinto: O(c). Después, la base de conocimiento registra la respuesta y aplica las reglas de dependencia (sección 4.9), que pueden resolver otros filtros sin preguntarlos.

Si el turno es un arriesgue fallido, se descarta sólo al personaje arriesgado. En los dos casos se verifica un invariante: el secreto del rival sigue entre los candidatos, así que el conjunto nunca queda vacío. Si quedara vacío, significaría que hubo respuestas contradictorias, y el programa lanza una excepción en lugar de seguir con datos imposibles.

En la interfaz, los descartados del último turno se marcan con borde cobre y los de turnos anteriores quedan tachados.

4.8 Criterio de parada y suposición final

Preguntar hasta que quede un solo candidato no siempre es lo mejor. Con k candidatos, arriesgar de a uno cuesta, en promedio:

$$E_{\text{arriesgar}}(k) = (1 + 2 + \dots + k) / k = (k + 1) / 2 \text{ turnos}$$

Si un arriesgue falla, ese personaje queda descartado y se arriesga el siguiente. La tabla compara esa opción con preguntar primero, usando la mejor división posible:

Candidatos (k)	Arriesgar de a uno: promedio / peor	Preguntar primero: promedio / peor	Conviene
1	1 / 1	—	Arriesgar
2	1,5 / 2	2 / 2	Arriesgar
3	2 / 3	2,33 / 3	Arriesgar
4	2,5 / 4	2,5 / 3	Preguntar: empatan en promedio y el peor caso es mejor

Por eso la máquina arriesga cuando le quedan **3 candidatos o menos** (`EstrategiaGreedy.UMBRAL_ARRIESGUE = 3`), y elige al azar el orden de los arriesgues. Esta regla no estaba en el primer diseño. La agregamos cuando la herramienta de análisis mostró que, con umbral 2, la máquina promediaba 5,17 turnos contra 5,04 del óptimo, y que la diferencia venía toda de los grupos de 3 candidatos.

Hay además un caso de seguridad: si ningún filtro separa a los candidatos, la máquina arriesga. Con un catálogo válido no pasa nunca, porque los perfiles son únicos.

4.9 Manejo de la dependencia lógica

Los filtros no son independientes. La clase `BaseConocimiento` guarda en un `EnumMap` cada filtro resuelto con su valor, y aplica dos reglas hasta que no se deduce nada nuevo:

- **Regla 1: grupo exclusivo del pelo.** Cada personaje cumple exactamente una de cuatro condiciones: es pelado, o tiene pelo colorado, negro o amarillo. Si una es SÍ, las otras tres son NO: un pelado no tiene color de pelo, y quien tiene pelo negro no es pelado ni tiene otro color. Si tres son NO, la cuarta es SÍ.
- **Regla 2: mujer implica sin barba.** Si ¿Es mujer? es SÍ, ¿Tiene barba? es NO. Por el contrarrecíproco, si ¿Tiene barba? es SÍ, ¿Es mujer? es NO.

Los filtros resueltos por inferencia no se vuelven a evaluar. La máquina no gasta turnos en preguntas cuya respuesta ya conoce, y para el humano esos botones se deshabilitan. En Máquina vs Máquina, cada inferencia aparece en la traza con su motivo, por ejemplo: "¿Tiene pelo negro? = NO (porque ¿Es pelado? es SÍ)".

Las reglas son $O(1)$: hay 8 filtros, cada pasada revisa a lo sumo 6 condiciones y la propagación termina en pocas pasadas. Si dos respuestas se contradicen, `BaseConocimiento` lanza una excepción. El `ValidadorCatalogo` garantiza que los datos cumplen las mismas reglas.

4.10 Precondiciones y validaciones implementadas

Dónde	Qué se valida	Si falla
ValidadorCatalogo	Hay exactamente 23 personajes y no hay nombres repetidos (sin distinguir tildes ni mayúsculas).	No se arma el tablero
ValidadorCatalogo	Pelado $\Leftrightarrow$ sin color de pelo; quien no es pelado tiene exactamente un color. Ninguna mujer tiene barba.	No se arma el tablero
ValidadorCatalogo	No hay dos personajes con el mismo perfil: todos son distinguibles.	No se arma el tablero
ValidadorCatalogo	El catálogo llega agrupado por género, como pide la consigna.	No se arma el tablero
Personaje	El nombre no está vacío. El ID se asigna una sola vez.	Excepción
Tablero	Cada personaje agregado va después del último. obtenerPorId recibe un ID entre 1 y 23.	Excepción
Jugador	El secreto es un personaje del tablero. No se pregunta un filtro ya resuelto. Los candidatos nunca quedan vacíos.	Excepción
BaseConocimiento	Dos respuestas sobre el mismo filtro no se contradicen.	Excepción
Partida	Cada jugador juega sólo en su turno y nadie juega después del final. Sólo se arriesgan personajes del tablero. Los secretos se revelan recién al terminar.	Excepción
Interfaz	El nombre arriesgado existe (búsqueda binaria). Los botones de filtros resueltos y los controles fuera de turno están deshabilitados.	Aviso al usuario

Las 54 pruebas automáticas cubren todas estas validaciones.

4.11 Tabla comparativa de tiempos

Medimos los cuatro ordenamientos sobre la misma lista de 23 personajes, en el orden del catálogo. Cada valor es el tiempo promedio de un ordenamiento, calculado así:

- Cada ordenamiento se repitió 20.000 veces, sobre copias de la lista preparadas de antemano.
- Antes de medir hubo una tanda de calentamiento, para que el compilador JIT de Java optimizara el código.
- Se reporta la mediana de 7 corridas y, al lado, el mínimo y el máximo.

Equipo: procesador AMD Ryzen 7 8845HS, Windows 11, JDK 25.

Algoritmo	Complejidad	Tiempo (ms)	Rango (ms)	Comparaciones
MergeSort	$\Theta(n \log n)$	0,00110	0,00104 – 0,00187	76
QuickSort	$O(n \log n)$ promedio	0,00071	0,00070 – 0,00118	118
Inserción	$O(n^2)$	0,00064	0,00063 – 0,00080	146
Burbujeo	$O(n^2)$	0,00118	0,00094 – 0,00167	232

Con 23 personajes, la diferencia no es significativa. Los cuatro tardan entre 0,6 y 1,2 microsegundos, e Inserción es incluso el más rápido. La diferencia entre el mejor y el peor es de medio microsegundo: miles de veces menos que lo que tarda la ventana en dibujar un solo cuadro (unos 16 ms). Además, el tablero se ordena una sola vez por partida.

La explicación está en las constantes que la notación O descarta. MergeSort hace menos comparaciones (76 contra 146 de Inserción), pero paga llamadas recursivas, copias al arreglo auxiliar y la conversión entre lista y arreglo. Con $n = 23$, $n \cdot \log_2 n \approx 104$ y $n^2 = 529$: la diferencia en cantidad de operaciones es chica, y el costo fijo de cada una pesa más. Inserción, además, trabaja en el lugar y aprovecha que la entrada no es completamente aleatoria.

La ventaja asintótica aparece cuando n crece. Estos son los mismos algoritmos sobre personajes al azar (mediana de 7 corridas, en ms):

n	MergeSort	QuickSort	Inserción	Burbujeo
23	0,0011	0,0007	0,0006	0,0012

n	MergeSort	QuickSort	Inserción	Burbujeo
1.000	0,091	0,063	1,047	5,588
10.000	2,43	2,16	138,4	692,4

Con 10.000 elementos, Inserción ya tarda 57 veces más que MergeSort, y Burbujeo 285 veces más: 24,8 y 50 millones de comparaciones contra 120 mil. Elegimos MergeSort no por su velocidad con 23 personajes, que en la práctica es la misma, sino por la garantía de peor caso, la estabilidad y porque el diseño no depende del tamaño del tablero. Con 1.000 personajes, el juego seguiría funcionando igual de bien.

5. Justificación de la estrategia Greedy utilizada

5.1 Por qué elegimos esta variante

Los cinco elementos de la técnica, como los vimos en clase:

Elemento	En nuestro juego
Conjunto de candidatos	Los filtros sin resolver.
Función de selección	El filtro con mayor descarte garantizado, $\min(\text{SÍ}, \text{NO})$.
Función de factibilidad	El filtro tiene que separar a los candidatos ($\text{descarte} > 0$) y no puede estar resuelto por la base de conocimiento.
Función objetivo	Adivinar el secreto en la menor cantidad de turnos.
Función solución	Quedan 3 candidatos o menos: a partir de ahí, se arriesga.

Consideramos otras funciones de selección:

- **Maximizar los SÍ**, es decir, preguntar por la característica más común. Descarta mucho cuando la respuesta es NO y casi nada cuando es SÍ: el resultado depende de la suerte.
- **Elegir al azar**. No aprovecha la información de los candidatos.
- **Máxima ganancia de información (entropía)**, como el algoritmo ID3 para árboles de decisión. Con preguntas de sí o no y candidatos igual de probables, también prefiere las divisiones más parejas, así que elige casi siempre lo mismo que nuestro criterio. Pero necesita logaritmos, y su resultado es más difícil de interpretar.

Elegimos el descarte garantizado por tres razones: protege contra la respuesta menos favorable, se calcula con conteos enteros y se explica en una frase: "pase lo que pase, esta pregunta descarta por lo menos a tantos".

5.2 Cómo se calcula el descarte

Para cada filtro sin resolver se recorren los c candidatos y se cuentan los que cumplen (SÍ). Los que no cumplen son $c - \text{SÍ}$. El descarte garantizado es el menor de los dos números. Después de la respuesta, el descarte real es el grupo que no coincidió, que es mayor o igual al garantizado.

5.3 Por qué es eficiente

Un turno cuesta $O(f \cdot c)$: con $f = 8$ filtros y $c \leq 23$ candidatos, menos de 200 evaluaciones de un atributo. La estrategia no mira hacia adelante ni vuelve atrás: decide con la información del turno y no guarda estados.

Una estrategia óptima, en cambio, tiene que considerar todas las secuencias posibles de preguntas y respuestas. Construir el árbol de decisión óptimo es un problema NP-completo en general (Hyafil y Rivest, 1976). Nuestra herramienta de verificación lo resuelve de forma exacta sólo porque 23 es un número chico: explora hasta $2^{23} \approx 8,4$ millones de subconjuntos de candidatos.

5.4 ¿La estrategia es óptima?

En nuestro tablero, sí. La herramienta [AnálisisOptimalidad](#) calcula la mejor estrategia posible con las mismas reglas (cualquier secuencia de preguntas y arriesgos), por búsqueda exhaustiva con memoria, y la compara con la nuestra:

Estrategia	Peor caso	Promedio
Greedy (la del juego, umbral de parada 3)	6 turnos	5,04 turnos
Óptima (búsqueda exhaustiva)	6 turnos	5,04 turnos
Greedy con umbral de parada 2 (primera versión)	6 turnos	5,17 turnos

En general, no. El Greedy mira un solo turno hacia adelante. La herramienta buscó subconjuntos del tablero donde pierde y encontró este, con 6 personajes:

Personaje	Características
Ana	Mujer, pelo amarillo, lentes
Elena	Mujer, pelada, lentes
Florencia	Mujer, pelo colorado, lentes
Hernán	Hombre, pelado, lentes, sombrero
Paula	Mujer, pelo negro, lentes, sombrero
Tomás	Hombre, pelo negro, barba

En el primer turno empatan tres filtros con descarte garantizado 2: ¿Es mujer? (4 SÍ, 2 NO), ¿Es pelado? (2 SÍ, 4 NO) y ¿Tiene pelo negro? (2 SÍ, 4 NO). El Greedy elige ¿Es mujer? por el orden de desempate. Si la respuesta es SÍ, quedan cuatro mujeres. ¿Es pelado? separa sólo a Elena, y quedan tres (Ana, Florencia y Paula) que se arriesgan de a uno: **5 turnos** en el peor caso.

La estrategia óptima pregunta primero ¿Es pelado?. Deja dos grupos, {Elena, Hernán} y {Ana, Florencia, Paula, Tomás}, que las preguntas siguientes parten por la mitad, y adivina siempre en **4 turnos**.

Por qué pasa. La función de selección mide cuánto se descarta ahora, pero no qué tan fácil de dividir es lo que queda. Tres preguntas que valen lo mismo en este turno dejan grupos que valen distinto en el siguiente. Para evitarlo habría que evaluar dos o más turnos hacia adelante, y eso es justamente dejar de ser Greedy.

Hay un segundo caso, con 4 personajes (Florencia, Gustavo, Julieta y Valentina), donde ningún filtro los parte en 2 y 2: todos dejan 3 de un lado y 1 del otro. Si pregunta, la máquina promedia 2,75 turnos; si arriesga de a uno desde el principio, 2,5. El criterio de parada cuenta candidatos, pero no tiene en cuenta que las preguntas disponibles sean malas.

6. Algoritmos no utilizados

Algoritmo visto en clase	Por qué no lo usamos
QuickSort	Está implementado, pero sólo para comparar. No lo usamos para el tablero porque su peor caso es $O(n^2)$ y no es estable (sección 4.2).
Inserción y Burbujeo	Son cuadráticos. Están en el proyecto sólo como referencia para la tabla de tiempos.
Dijkstra, Prim y Kruskal	Resuelven problemas sobre grafos con pesos: caminos mínimos y árboles de recubrimiento mínimo. En el juego no hay grafo ni costos entre personajes.
Código de Huffman	Construye el árbol de prefijos de largo promedio mínimo cuando cualquier división de los símbolos está permitida. Es muy cercano a nuestro problema: si se pudiera preguntar "¿está en este grupo?" por cualquier grupo de personajes, Huffman daría el árbol de preguntas óptimo en promedio. No aplica porque sólo se puede preguntar por los 8 filtros, que dividen a los candidatos de formas fijas.

Algoritmo visto en clase	Por qué no lo usamos
Mochila fraccionaria y problema del cambio	Optimizan una suma de valores con una capacidad o un monto. En el juego no hay pesos, valores ni montos que repartir.
Matrimonios estables	Empareja dos grupos según sus preferencias. En el juego no hay nada que emparejar.
Otros ejemplos de Divide y Conquista (torres de Hanoi, elemento mayoritario, suma parcial máxima, fixture de torneo, potencia rápida)	Resuelven problemas que el juego no tiene. De esa familia sí usamos la búsqueda binaria, para el arriesgue.
Programación dinámica	Todavía no la vimos en la materia. La herramienta de verificación usa una búsqueda exhaustiva con memorización, que es la idea de la programación dinámica, para calcular la estrategia óptima. No está en el juego: necesita memoria para 2^{23} estados, y con más personajes se vuelve impracticable, mientras que el Greedy decide cada turno en $O(f \cdot c)$.

7. Notación Big O para nuestra app

Operación	Clase	Complejidad
Validar el catálogo	ValidadorCatalogo	$O(n)$
Ordenar el tablero	MergeSort	$\Theta(n \log n)$
Asignar IDs y agregar	Tablero	$O(1)$ cada uno, $O(n)$ en total
Buscar un personaje por ID	Tablero	$O(1)$
Buscar un personaje por nombre (arriesgue)	BusquedaBinaria	$O(\log n)$
Evaluar los filtros y elegir uno	EstrategiaGreedy	$O(f \cdot c)$
Descartar candidatos	Jugador	$O(c)$
Inferir por dependencia lógica	BaseConocimiento	$O(1)$
Responder una pregunta	Oraculo (Jugador)	$O(1)$
Construir el árbol de decisión	ArbolDecision	$O(f \cdot n \cdot h)$

Una partida tiene dos partes. La **preparación** valida y ordena el tablero: $O(n) + \Theta(n \log n) = \Theta(n \log n)$. Después vienen los **turnos**: cada turno de la máquina cuesta $O(f \cdot c) \leq O(f \cdot n)$, y la cantidad de turnos t está acotada por la altura del árbol de decisión más los arriesgues finales.

$$T(\text{partida}) = \Theta(n \log n) + t \cdot O(f \cdot n)$$

¿Cuánto vale t ? En el peor caso teórico, con filtros que separaran a un solo personaje por vez, cada pregunta descartaría uno y t crecería como n : la partida sería $O(f \cdot n^2)$. Ese caso no puede darse en nuestro juego. La cantidad de filtros es fija ($f = 8$) y cada uno se pregunta una vez como mucho, así que la máquina hace a lo sumo 8 preguntas y 3 arriesgues. Y con filtros que dividen parejo, como los nuestros, t crece como $\log_2 n$: en el tablero real, la máquina adivina en 6 turnos como máximo.

Con f constante y t proporcional a $\log n$, los turnos cuestan $O(n \log n)$, igual que la preparación. Por eso la notación más certera para el juego es **$O(n \log n)$** : el ordenamiento inicial y el recorrido del árbol de decisión crecen al mismo ritmo, y ninguna parte del programa es cuadrática. La interfaz no cambia el análisis, porque dibujar el tablero es $O(n)$ por cuadro.

8. Reflexión sobre el TP

Logros

- La máquina adivina a cualquiera de los 23 personajes en 6 turnos como máximo y en 5,04 en promedio. En este tablero, eso es exactamente el óptimo.
- El secreto del humano está protegido por diseño: la máquina no tiene ninguna forma de leerlo.

- Todo lo que afirma este documento se puede reproducir, porque las pruebas, las mediciones y el análisis de optimalidad son programas del proyecto.
- El modo Máquina vs Máquina muestra cada decisión con sus números, así que el algoritmo se puede seguir y explicar turno por turno.

Dificultades

- Los filtros de la consigna no alcanzaban para distinguir a 23 personajes.
- Medir de forma confiable tiempos de un microsegundo.
- Encontrar un contraejemplo concreto para el Greedy. En el tablero completo empata al óptimo, así que hubo que buscarlo en subconjuntos.
- La ventaja de empezar: en 200 partidas Máquina vs Máquina con secretos al azar, la máquina que empieza ganó 130 (65 %).

Propuestas de mejora

- Desempatar la función de selección mirando un turno más adelante, para evitar el contraejemplo de la sección 5.4.
- Un criterio de parada que compare el costo esperado de preguntar y de arriesgar con los candidatos concretos, en lugar de un umbral fijo.
- Niveles de dificultad; por ejemplo, una máquina "fácil" que elija filtros al azar.
- Cargar el catálogo desde un archivo, para jugar con otros personajes sin recompilar.
- Un modo en el que el humano conteste a mano, con detección de respuestas contradictorias (la base de conocimiento ya las detecta).
- Pasar las pruebas a JUnit.

[COMPLETAR: reflexión personal del equipo]

9. Bibliografía, fuentes y ayudas

- Consigna "TP – Adivina Quién" y "Documentación técnica – Trabajo práctico obligatorio". Diseño y Análisis de Algoritmos, prof. Juan Ignacio López, UADE, 2026.
- Diapositivas de la cátedra de Programación III (UADE): notación Big O, Divide y Conquista y algoritmos Greedy.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. Introduction to Algorithms, 3.^a ed. MIT Press, 2009. Capítulos 2 (MergeSort), 4 (teorema maestro), 7 (QuickSort) y 16 (algoritmos greedy y Huffman).
- Hyafil, L. y Rivest, R. L. "Constructing optimal binary decision trees is NP-complete". Information Processing Letters 5(1), 1976, pp. 15–17.
- Quinlan, J. R. "Induction of decision trees". Machine Learning 1, 1986, pp. 81–106 (algoritmo ID3).
- Documentación oficial de Java SE 17: java.util (ArrayList, LinkedHashSet, EnumMap, EnumSet) y javax.swing.
- Claude (Anthropic), a través de Claude Code: asistente de IA usado para el diseño, el código, las pruebas y la redacción (ver 3.3).
- **[COMPLETAR: otras fuentes que haya usado el equipo]**

Anexo · Capturas de la aplicación

Capturas de las pantallas del juego. No cuentan para la extensión del documento.

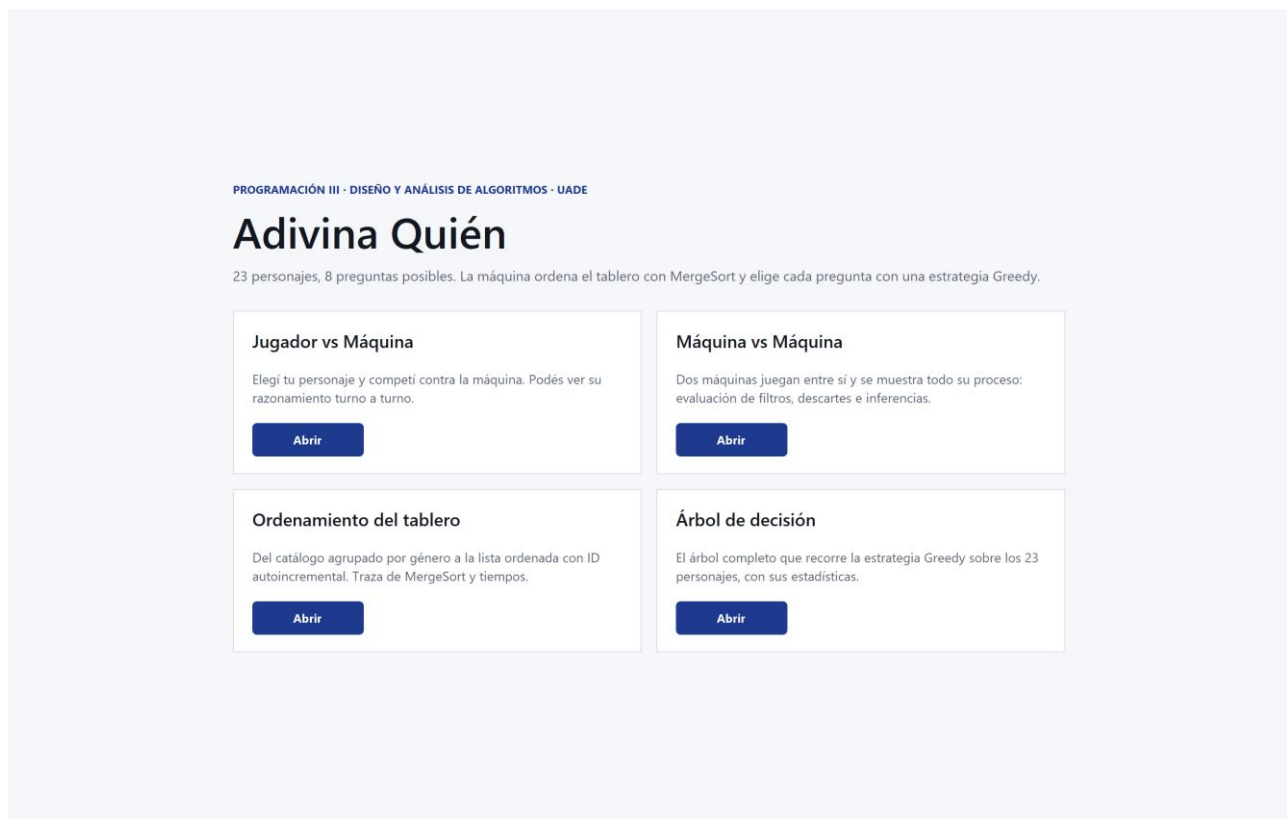


Figura 16. Menú principal.

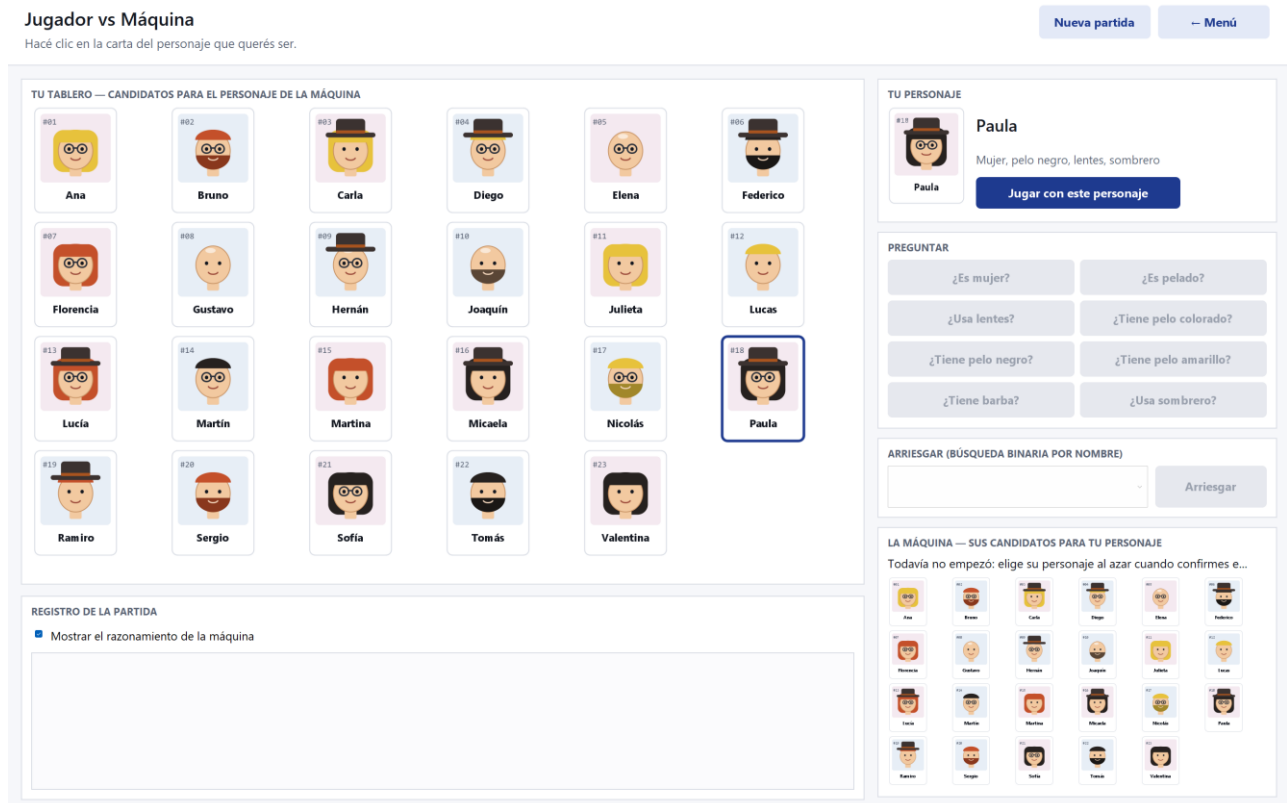


Figura 17. Jugador vs Máquina: el humano elige su personaje haciendo clic en una carta.

Jugador vs Máquina

Tu turno: hacé una pregunta o arriesgá.

Nueva partida

— Menú

TU TABLERO — CANDIDATOS PARA EL PERSONAJE DE LA MÁQUINA



TU PERSONAJE



PREGUNTAR

¿Es mujer? ¿Es pelado?

¿Usa lentes? ¿Tiene pelo colorado?

¿Tiene pelo negro? ¿Tiene pelo amarillo?

¿Tiene barba? ¿Usa sombrero?

ARRIESGAR (BÚSQUEDA BINARIA POR NOMBRE)

LA MÁQUINA — SUS CANDIDATOS PARA TU PERSONAJE

A la máquina le quedan 6 candidatos. A vos, 5.



REGISTRO DE LA PARTIDA

☒ Mostrar el razonamiento de la máquina

La máquina armó el tablero (MergeSort por nombre, IDs 1 a 23) y eligió su personaje al azar.
Empezás vos.

— Turno 1 · Vos —
Pregunta: ¿Es mujer? -> SÍ
Descartados (12): [Bruno, Diego, Federico, Gustavo, Hernán, Joaquín, Lucas, Martín, Nicolás, Ramiro, Sergio, Tomás]
Quedan 11 candidatos

— Turno 2 · Máquina —

Figura 18. Jugador vs Máquina en juego. A la izquierda, los candidatos del humano; abajo a la derecha, los de la máquina (el personaje del humano, en dorado). Los botones de filtros resueltos se deshabilitan.

Máquina vs Máquina

Turno 6 · Máquina B

Velocidad

Automático

Siguiente turno

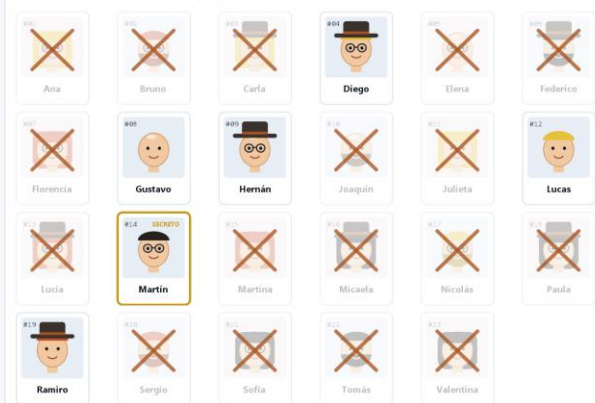
Nueva partida

— Menú

Máquina A — su personaje: Martín
Busca a Florencia (marcado en dorado, sólo lo ve el espectador)
Candidatos: 2 de 23 · preguntas hechas: 3



Máquina B — su personaje: Florencia
Busca a Martín (marcado en dorado, sólo lo ve el espectador)
Candidatos: 6 de 23 · preguntas hechas: 2



PROCESO DE LA MÁQUINA, TURNO POR TURNO

Tablero ordenado con MergeSort por nombre (IDs 1 a 23).
Máquina A tiene a Martín; Máquina B tiene a Florencia.
Ninguna ve el secreto de la otra: sólo le hace preguntas a través de la interfaz Oráculo.
Empieza la Máquina A.

— Turno 1 · Máquina A —
Candidatos antes: 23
Evaluación Greedy (descarte garantizado = min(SÍ, NO)):
Filtro SÍ NO descarte
¿Es mujer? 11 12 11 <- elegido
¿Es pelado? 4 19 4

Figura 19. Máquina vs Máquina: cada tablero muestra los candidatos de una máquina. Abajo, la traza con la tabla de evaluación del Greedy en cada turno.

Ordenamiento del tablero

La máquina recibe los personajes agrupados sólo por género y los ordena por nombre con MergeSort (Divide y Conquista).

[← Menú](#)
Del catálogo al tablero **Traza de MergeSort** Comparación de tiempos

1 - CATÁLOGO INICIAL: AGRUPADO POR GÉNERO, SIN OTRO ORDEN

Posición	Nombre	Género
1	Sofía	Mujer
2	Carla	Mujer
3	Lucía	Mujer
4	Valentina	Mujer
5	Ana	Mujer
6	Martina	Mujer
7	Paula	Mujer
8	Julieta	Mujer
9	Florencia	Mujer
10	Elena	Mujer
11	Micaela	Mujer
12	Tomás	Hombre
13	Diego	Hombre
14	Joaquín	Hombre
15	Bruno	Hombre
16	Martín	Hombre
17	Hernán	Hombre
18	Lucas	Hombre
19	Ramiro	Hombre
20	Gustavo	Hombre
21	Federico	Hombre
22	Nicolás	Hombre
23	Sergio	Hombre

2 - TABLERO: MERGESORT POR NOMBRE, ID AUTOINCREMENTAL AL AGREGAR

ID	Nombre	Género	Pelo	Lentes	Barba	Sombrero
1	Ana	Mujer	Amarillo	Sí	No	No
2	Bruno	Hombre	Colorado	Sí	Sí	No
3	Carla	Mujer	Amarillo	No	No	Sí
4	Diego	Hombre	Amarillo	Sí	No	Sí
5	Elena	Mujer	Sin pelo	Sí	No	No
6	Federico	Hombre	Negro	No	Sí	Sí
7	Florencia	Mujer	Colorado	Sí	No	No
8	Gustavo	Hombre	Sin pelo	No	No	No
9	Hernán	Hombre	Sin pelo	Sí	No	Sí
10	Joaquín	Hombre	Sin pelo	No	Sí	No
11	Julieta	Mujer	Amarillo	No	No	No
12	Lucas	Hombre	Amarillo	No	No	No
13	Lucía	Mujer	Colorado	Sí	No	Sí
14	Martín	Hombre	Negro	Sí	No	No
15	Martina	Mujer	Colorado	No	No	No
16	Micaela	Mujer	Negro	No	No	Sí
17	Nicolás	Hombre	Amarillo	Sí	Sí	No
18	Paula	Mujer	Negro	Sí	No	Sí
19	Ramiro	Hombre	Colorado	No	No	Sí
20	Sergio	Hombre	Colorado	No	Sí	No
21	Sofía	Mujer	Negro	Sí	No	No
22	Tomás	Hombre	Negro	No	Sí	No
23	Valentina	Mujer	Negro	No	No	No

Figura 20. Del catálogo agrupado por género al tablero ordenado por MergeSort, con ID autoincremental.

Ordenamiento del tablero

La máquina recibe los personajes agrupados sólo por género y los ordena por nombre con MergeSort (Divide y Conquista).

[← Menú](#)
Del catálogo al tablero **Traza de MergeSort** Comparación de tiempos

```

MergeSort(catálogo, criterio = nombre sin tildes)
T(n) = 2·T(n/2) + O(n) => Θ(n log n) · n = 23: 22 divisiones, 22 combinaciones

dividir [0..22] en [0..11] y [12..22]
  dividir [0..11] en [0..5] y [6..11]
    dividir [0..5] en [0..2] y [3..5]
      dividir [0..2] en [0..1] y [2..2]
        dividir [0..1] en [0..0] y [1..1]
          combinar [0..1] -> [Carla, Sofía]
        combinar [0..2] -> [Carla, Lucía, Sofía]
      dividir [3..5] en [3..4] y [5..5]
        dividir [3..4] en [3..3] y [4..4]
          combinar [3..4] -> [Ana, Valentina]
        combinar [3..5] -> [Ana, Martina, Valentina]
      combinar [0..5] -> [Ana, Carla, Lucía, Martina, Sofía, Valentina]
    dividir [6..11] en [6..8] y [9..11]
      dividir [6..8] en [6..7] y [8..8]
        dividir [6..7] en [6..6] y [7..7]
          combinar [6..7] -> [Julieta, Paula]
        combinar [6..8] -> [Florencia, Julieta, Paula]
      dividir [9..11] en [9..10] y [11..11]
        dividir [9..10] en [9..9] y [10..10]
          combinar [9..10] -> [Elena, Micaela]
        combinar [9..11] -> [Elena, Micaela, Tomás]
      combinar [6..11] -> [Elena, Florencia, Julieta, Micaela, Paula, Tomás]
    combinar [0..11] -> [Ana, Carla, Elena, Florencia, Julieta, Lucía, Martina, Micaela, Paula, Sofía, Tomás, Valentina]
  dividir [12..22] en [12..17] y [18..22]
    dividir [12..17] en [12..14] y [15..17]
      dividir [12..14] en [12..13] y [14..14]
        dividir [12..13] en [12..12] y [13..13]
          combinar [12..13] -> [Diego, Joaquín]
        combinar [12..14] -> [Bruno, Diego, Joaquín]
      dividir [15..17] en [15..16] y [17..17]
        dividir [15..16] en [15..15] y [16..16]
          combinar [15..16] -> [Hernán, Martín]
        combinar [15..17] -> [Hernán, Lucas, Martín]
      combinar [12..17] -> [Bruno, Diego, Hernán, Joaquín, Lucas, Martín]
    dividir [18..22] en [18..20] y [21..22]
      dividir [18..20] en [18..19] y [20..20]
        dividir [18..19] en [18..18] y [19..19]
          combinar [18..19] -> [Gustavo, Ramiro]
        combinar [18..20] -> [Federico, Gustavo, Ramiro]
      dividir [21..22] en [21..21] y [22..22]
        combinar [21..22] -> [Micaela, Sofía]

```

Figura 21. Traza de MergeSort: cada división y cada combinación.

Ordenamiento del tablero

[← Menú](#)

La máquina recibe los personajes agrupados sólo por género y los ordena por nombre con MergeSort (Divide y Conquista).

Del catálogo al tablero **Traza de MergeSort** Comparación de tiempos**Medir tiempos**

Mide MergeSort, QuickSort, Inserción y Burbujeo sobre los mismos 23 personajes, y sobre 1.000 y 10.000 al azar.

Algoritmo	Complejidad	n	Repeticiones	ms por ordenamiento	Comparaciones
MergeSort	$\Theta(n \log n)$	23	20.000	0,00168	76
QuickSort	$O(n \log n)$ prom. / $O(n^2)$ peor	23	20.000	0,00126	118
Inserción	$O(n^2)$	23	20.000	0,00080	146
Burbujeo	$O(n^2)$	23	20.000	0,00103	232
MergeSort	$\Theta(n \log n)$	1.000	100	0,12567	8.707
QuickSort	$O(n \log n)$ prom. / $O(n^2)$ peor	1.000	100	0,11642	10.660
Inserción	$O(n^2)$	1.000	100	0,91300	251.664
Burbujeo	$O(n^2)$	1.000	100	4,93988	499.175
MergeSort	$\Theta(n \log n)$	10.000	5	2,35296	120.384
QuickSort	$O(n \log n)$ prom. / $O(n^2)$ peor	10.000	5	2,07828	155.631
Inserción	$O(n^2)$	10.000	5	120,55588	24.817.574
Burbujeo	$O(n^2)$	10.000	5	609,11568	49.993.289

Con $n = 23$ las diferencias son de microsegundos: menos que el ruido del sistema y mucho menos de lo que tarda en dibujarse la pantalla. El conteo de comparaciones muestra la tendencia ($n \log n$ contra n^2) que con 10.000 elementos el reloj.

Figura 22. Comparación de tiempos en la aplicación. Son los de una sola corrida; la tabla de la sección 4.11 usa la mediana de 7.